

Remote Procedure Call (RPC)

Grundidee

RPC erlaubt das Aufrufen einer Funktion auf einem entfernten Rechner, als wäre sie lokal.

Ablauf: Client → Request → Server

Server → Response → Client

Eigenschaften: - basiert auf Request-Reply Kommunikation - meist synchrone Kommunikation - Parameter werden marshalled / unmarshalled - häufig **at-least-once Semantik** - Datencodierung z.B. XDR

Ablauf eines RPC

1. Client ruft Funktion auf
 2. Client-Stub erzeugt Request
 3. Nachricht wird über Netzwerk gesendet
 4. Server-Stub empfängt Nachricht
 5. Parameter werden dekodiert
 6. Server führt Methode aus
 7. Ergebnis wird zurückgesendet
-

RPC Registrierung

Server registriert RPC-Service bei einem Namensdienst (Portmapper / rpcbind).

Server → Registrierung

Client → lookup(service)

Client → RPC Call

Stub und Skeleton

Low-Level RPC ist komplex → automatische Generierung.

Client Stub - lokale Schnittstelle für Client - marshalled

Parameter - sendet Request

Server Skeleton - empfängt Request - unmarshalled Parameter - ruft Serverimplementierung auf

Moderne RPC Technologien

gRPC

Eigenschaften:

- nutzt Protocol Buffers
- Transport über HTTP/2
- Streaming möglich
- Multi-Language Support
- TLS Verschlüsselung

Einsatz:

- Microservices
 - Cloud Systeme
 - Mobile Backends
-

Verteilte Objektkommunikation

Grundidee

Objekte können über Prozess- oder Rechengrenzen hinweg interagieren.

Lokal Entfernt

direkter Methodenaufruf
Netzwerkcommunication
gleiche Speicheradresse
RPC / Messaging

Entfernte Objektreferenz

Identifiziert ein Objekt eindeutig.

Bestandteile:

- IP-Adresse

- Port
 - Zeitstempel
 - Objektnummer
 - Schnittstelle
-

Entfernte Schnittstellen

Definieren Methoden eines entfernten Objekts.

Enthalten:

- Interface Name
 - Datentypen
 - Methodensignaturen
 - Parameter
 - Rückgabewerte
-

Architektur verteilter Objekte

Client

↓

Proxy / Stub

↓

Kommunikationsschicht

↓

Skeleton

↓

Serverobjekt

Proxy: - implementiert entfernte Schnittstelle - marshalled Requests

Skeleton: - unmarshalled Requests - ruft Servermethode auf

Dispatcher: - wählt passende Methode

Parameterübergabe

Kopiersemantik (Pass by Value)

- Objekt wird serialisiert

- Kopie wird übertragen
- Client erhält neue Instanz

Referenzsemantik (Pass by Reference)

- Client erhält Stub auf Serverobjekt
 - weitere Aufrufe gehen über Netzwerk
-

Weitere Aspekte verteilter Objekte

Infrastruktur:

- Namensdienst
- Threading für parallele Requests
- Aktivierung von Objekten
- persistenter Objektspeicher
- verteilte Garbage Collection

Garbage Collection Problem: Referenzen können auch in Nachrichten existieren.

Lösungen: - Ping - Lease Mechanismen

RMI (Remote Method Invocation)

Java-Technologie für verteilte Objekte.

Komponenten:

- Remote Interface
- Server Implementation
- RMI Registry
- Client

Client findet Objekt über:

lookup()

Namensdienste

Definition

Namensdienst bildet

Name → Zugriffsinformation

Operationen:

- bind(name, object)
 - lookup(name)
 - rebind(name, object)
-

Vorteile von Namensdiensten

- Ortstransparenz
 - Ressourcen können verschoben werden
 - Lastverteilung
 - Fehlertoleranz
 - keine hardcodierten Adressen
-

Namensräume und Kontext

Ein Kontext enthält:

- Namensraum
- Bindings Name → Resource

Hierarchische Struktur möglich:

Kunde / Privatkunde / PeterMeier

Federation von Namensdiensten

Mehrere Namensdienste können zusammenarbeiten.

Beispiel:

clemens@domain.de

= Benutzername + Domainname

Beispiele für Namensdienste

- DNS
 - LDAP
 - RPC Registry
 - RMI Registry
 - CORBA Naming Service
-

URI / URL

URI = allgemeiner Ressourcenname

Beispiel:

<http://example.org/resource>

Enthält:

- Technologie (http, rmi)
 - Domain
 - hierarchischen Pfad
-

Verzeichnisdienste

Erweiterung von Namensdiensten.

Speichern zusätzlich:

Metadaten über Ressourcen

Beispiele:

- LDAP
 - NIS
 - UDDI
-

LDAP

Eigenschaften:

- hierarchischer Objektbaum

- Attribute-Value Paare
- flexible Schemas

Beispiel:

cn=John, ou=Marketing, ou=East

JNDI

Java Naming and Directory Interface

API für Zugriff auf:

- DNS
- LDAP
- RMI
- CORBA

Beispiel:

```
Context ctx = new InitialContext();  
Object obj = ctx.lookup("service");
```

Wichtige Klausurpunkte

RPC - entferntes Verfahren wie lokale Funktion

Stubs / Skeletons - abstrahieren Netzwerkkommunikation

Verteilte Objekte - Zugriff über entfernte Referenzen

Namensdienste - Name → Zugriffsinformation

Verzeichnisdienste - Namensdienst + Metadaten